A Deterministic Witness Rule for iPRA

Equivalence, complete checkers, and the limits of the reduction

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

The independent choice between case introduction and equation-progress introduction in the supplied paired-binder calculus is admissibly removed. Progress is selected exactly when the matrix is a literal equation and the existential witness is the literal zero term; every other introduction uses cases. An unused bound variable recovers progress at other witnesses. The independently certified term $C = I(2, 1; v.0) = 0$ recovers atomic cases at zero without presupposing substitution admissibility for the new rule. The resulting calculus preserves every native sequent through the identity on formulas, and hence preserves the inherited whole-domain bi-interpretation and equiconsistency with first-order intuitionistic PRA.

The complete new checker has 555 normalized Python bytes. A separate implementation of the unchanged Choice-DI calculus has 550 bytes. The inspected current corpus attains a minimum of 558 bytes, with different programs attaining 262 tokens and 296 AST nodes. Our two selected programs do not improve those latter minima. The code reduction comes from a small dispatch simplification and an inherited invalid-image convention; the new deterministic rule costs five bytes relative to the 550-byte control. These contributions are accounted for separately.

This is a verified small reduction, not the requested significant new arithmetic foundation or a proof of absolute simplicity. The package contains written proofs, complete source, executable proof normalization, adversarial controls, the frozen source census, and a compiled report. The mathematical metatheorems are not Lean or Rocq formalizations.

Established: one deterministic introduction pattern, the same native consequence relation, two smaller complete byte profiles, and a separate restricted-congruence theorem.
Not established: a global minimum, a simultaneous improvement of all size coordinates, or the requested major foundational breakthrough.

Contents

1 What changed, and what the comparison actually shows **3**

2 A deterministic introduction rule selected by the witness **4**

3 The complete checker and its exact trust boundary **9**

4 Why this is full intuitionistic PRA **11**

5 Reproducibility, evidence, and remaining scope **12**

Attributed complete bridge proofs **14**

A Complementary introductions with one local assumption **14**

B Internal scalar predecessor bootstrap **19**

C From literal-equation induction to full intuitionistic PRA **23**

D Pairing, primitive recursion, and internal numerical composites **29**

E Arithmetic strength and both ordinary interpretation composites **30**

F A narrower congruence rule and its exact boundary **33**

1 What changed, and what the comparison actually shows

The research target is full first-order intuitionistic primitive recursive arithmetic, with the symbols of fixed finite primitive-recursive descriptions, their defining equations, the successor axioms, and quantifier-free induction in intuitionistic first-order equality logic. Descriptions are external finite syntax. A number does not range over arbitrary functions, and no uniformly total interpreter is postulated. The requirement here includes internally provable interpretation composites, not merely equiconsistency or agreement on standard numerals.

The selected repository snapshot is `76a2651601d17b34d46753f189ea026a21ce8d29` [1]. It contains 17 direct ZIPs in `docs/drafts`. Recursive inspection finds 32 distinct ZIP payloads, 443 distinct Python source byte strings, and 2,239 source occurrences after archive-content deduplication. Every source at or below at least one observed boundary was inspected: 36 candidates comprise three complete checkers and 33 supporting utilities. The full source census and all 443 frozen source byte strings are supplied. The historical sources are parsed for measurement, not executed by the census.

The current byte minimum is the certified-zero program at 558 bytes. Its distinct joint profile attains 262 significant tokens and 296 AST nodes. These are the smallest values *attained in this finite collection*. They are upper bounds on an unknown global optimum, not mathematical lower bounds on every possible axiomatization or checker.

1.1 Three contributions that must not be conflated

A rule-normalization theorem. The new calculus $\mathcal{C}_{\mathrm{w}}$ has one introduction pattern whose choice is determined by the witness syntax. It retains the same numerical operations, formula meanings, equality transport, and elimination rules. The proof in Section 2 preserves all native sequents. Its main technical point is to use hypothesis cut, not an unjustified numerical substitution that could change the syntactic guard.

A small implementation improvement. Every selected major triple is truthy, and every generated computation equation is a nonempty array. A Boolean expression therefore shares their dispatch. Canonical null markers are replaced by empty lists. This saves six normalized bytes with no change of native theory. The inherited invalid image $0j$ saves another two bytes relative to `None`. Thus the unchanged Choice-DI theory has a complete 550-byte implementation. Adding the deterministic witness guard gives the 555-byte implementation of $\mathcal{C}_{\mathrm{w}}$. The mathematical restriction alone is not credited with the byte saving.

An independent congruence refinement. Appendix F shows that counter congruence for a different, counter-dependent recursor needs only seed zero and state-free updates. The displayed numerical-congruence derivation adds twenty nodes, or five for a distinct wrapped rule. This result does not implement the measured scalar checker and is assigned no byte record.

1.2 Exact whole-program sizes

Complete program	Bytes	Tokens	AST	Gzip-9
Supplied certified-zero byte profile	558	264	310	345
Supplied certified-zero joint profile	598	262	296	350
Same byte baseline, invalid image only	556	264	310	343
New unchanged-choice implementation	550	264	310	342
New deterministic-witness implementation	555	266	312	344
New witness implementation, explicit type	559	266	312	348
New witness implementation, explicit list	563	266	310	349

The two smallest new files are `code/checker_choice.py` and `code/checker.py`. The last two rows use an ordinary exact-type test and `isinstance`, respectively; all four avoid the generic-iterable malformed-string qualification of the old joint profile. The 550-byte reduction is $8/558 \approx 1.43\%$; the 555-byte reduction is $3/558 \approx 0.54\%$. No selected program is below all three old coordinate minima. The readable expansion is separately measured in `evidence/measurements.json`, rather than omitted from accounting or mistaken for a dependency of the compact file.

Measurements include imports, input loading, scope checking, computation, logic, and the acceptance verdict. Python and its standard JSON library are the common runtime boundary used by the comparison programs. Exact measurements use CPython 3.12.14 and zlib 1.3.2, normalize by `ast.unparse(ast.parse(raw)) + LF`, count all `ast.walk` nodes, and exclude only comment/layout/encoding tokens from token counts. No axiom data or validator is hidden outside a measured program. Gzip is `gzip.compress(raw, compresslevel=9, mtime=0)`, including its wrapper; it is an informational figure and not an optimization target.

2 A deterministic introduction rule selected by the witness

The independent case/progress choice can be removed from paired introduction without changing the native consequence relation. The replacement selects progress precisely when the matrix is a literal numerical equation and the supplied witness is the literal zero term. Every other introduction uses the zero case. This is a restriction on finite syntax, not a test of the value denoted by a term.

2.1 The two precisely specified calculi

Numerical terms are finite expressions

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h),$$

where only v in h is bound. Every update is fixed finite syntax with passive parameters, and all displayed substitutions avoid capture. For the executable interface we use the conservative closure extension of the Eq/D language:

$$A ::= (s = t) \mid D_{x,y}A \mid Q_yA.$$

The primary Eq/D fragment is recovered by replacing Q_yB with $D_{u,y}B$ for an unused u. The two expressions are interderivable: paired elimination gives the closure in one direction, and case introduction with the unused witness $S0$, using its base and QE, gives the other. The inherited logical bridge explains this conservative notation. The ordinary reading of a closure is $B(0) \to \forall y\, B(y)$; the ordinary

reading of $D_{x,y}B$ is $\exists x\,(B(x,0) \to \forall y\, B(x,y))$. Neither reading is changed by the construction below. Contexts are finite reusable lists; auxiliary closures may occur as local resources.

The two calculi have the same computation axioms. The proof works with the unrestricted iterator equations, and also with the smaller certified-zero inventory

$$\begin{aligned} I(0,a;v.h) &= a && \text{when } a \not\equiv_{\mathrm{syn}} 0,\\ I(Sn,a;v.h) &= h[I(n,a;v.h)/v] && \text{when } n, a \not\equiv_{\mathrm{syn}} 0. \end{aligned}$$

Here $\equiv_{\mathrm{syn}}$ denotes literal syntactic identity. Computation axioms may be leaves in every context. Equivalently, a checker may supply such an axiom as a local assumption and subsequently use hypothesis closure; the cut lemma below justifies the equivalence.

The common logical rules are exactly hypothesis use, equality transport, closure specialization, and paired elimination:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)},$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(r)}, \qquad \frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A}.$$

The numerical parameter b in paired elimination is fresh for the retained context, conclusion, and passive matrix parameters. The transport motive F is any fixed native formula template, including one whose hole lies beneath a binder; binders are first renamed away from the substituted terms.

Let $\mathcal{C}$ additionally have the two inherited introductions

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \text{(case)}, \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)}\ \text{(progress)}, \tag{$\mathrm{DI_{old}}$}$$

where E must be a literal equation. In both rules z is fresh for Γ, the witness t, and every passive matrix parameter.

For the new calculus $\mathcal{C}_{\mathrm{w}}$ define, on fixed syntax,

$$\delta_{B,t}(z) = \begin{cases} z, & B \text{ is a literal equation and } t \equiv_{\mathrm{syn}} 0,\\ 0, & \text{otherwise.} \end{cases}$$

Its only paired-introduction pattern is

$$\frac{\Gamma, B(t, \delta_{B,t}(z)) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}. \tag{$\mathrm{DI_w}$}$$

It has the same freshness conditions. In particular, a nonliteral term denoting zero is still a case witness. Compound matrices always use cases. No additional induction, substitution, reflexivity, symmetry, or matrix-congruence rule is assumed.

2.2 Structural facts established before the simulation

Lemma 2.1 (Renaming, weakening, and ordinary-assumption cut)**.** *In $\mathcal{C}_{\mathrm{w}}$, injective renaming of numerical variables, weakening of reusable contexts, and ordinary-assumption cut are admissible. The latter has the form*

$$\frac{\Gamma \vdash A \qquad \Gamma, A \vdash H}{\Gamma \vdash H},$$

where A is an object formula. The same conclusions hold with additional retained local closure clauses.

Proof. Variable renaming leaves both tests in ($\mathrm{DI_w}$) unchanged: a literal equation remains a literal equation, and renaming never turns a nonzero syntactic term into the literal constant 0, or conversely. It also preserves the syntactic conditions on computation axioms. Structural induction on a derivation therefore proves renaming admissible. Injections on the finitely many variables occurring in a proof suffice; fresh variables can always be chosen outside another fixed finite derivation.

For weakening, first rename local eigenparameters away from the added context, then retain that context throughout the original proof. Every inference is still an instance of the same rule, with the same witness syntax. Exchange and contraction are harmless because contexts are reusable lists and hypothesis closure is membership.

For cut, induct on the derivation of $\Gamma, A \vdash H$. At a use of the cut assumption, insert the given derivation of A, weakened by any local assumptions. Other hypotheses and computation leaves are unchanged. Transport and closure specialization commute with this replacement. At a paired elimination or introduction, first rename its eigenparameter away from the finite proof being inserted. Apply the induction hypothesis to its premise derivation or derivations and repeat the original inference. This operation substitutes derivations for hypothesis uses; it never substitutes numerical terms into the witness field. Consequently the literal-zero test and the literal-equation test are preserved at every introduction. This proves cut without assuming admissibility of arbitrary numerical substitution. □

Lemma 2.2 (A closed nonliteral representative of zero)**.** *Both calculi prove $C = 0$ and $0 = C$, where*

$$C := I(S(S0), S0; v.0).$$

These proofs use only permitted computation axioms and equality transport, even with the restricted computation inventory above.

Proof. The successor computation with predecessor $S0$ and initializer $S0$ gives $C = 0$ directly: both supplied terms are nonliteral zero, and the constant update returns 0.

To obtain the reverse equation without presupposing symmetry or reflexivity, put $K := I(0, C; v.0)$. The permitted base computation gives $K = C$, since its initializer C is not the literal zero term. Apply equality transport along $K = C$ to a second copy of $K = C$, with motive $F(w) := (w = C)$. The conclusion is $C = C$. Now transport this conclusion along $C = 0$, with the same motive, obtaining $0 = C$.

Thus neither a base computation with literal-zero initializer nor an introduction rule is used in the construction. If desired, $0 = 0$ follows separately by transporting $C = 0$ along itself with motive $w = 0$; it is not needed as a hidden premise of the displayed proof. □

Lemma 2.3 (Computation repair independent of introduction)**.** *Every unrestricted iterator computation equation is derivable from the restricted computation inventory by equality transport alone. No paired introduction, ordinary-assumption cut, or admissible numerical substitution is needed for this assertion.*

Proof. The theorem $C = 0$ was obtained directly in Lemma 2.2. A base equation with initializer literal zero is repaired using the fixed motive

$$F(w) := \big(I(0, w; v.h) = w\big).$$

Its instance $F(C)$ is a permitted computation axiom. Transport along $C = 0$ gives $F(0)$, the missing computation.

For a successor computation, replace each forbidden literal-zero supplied field by the same fresh placeholder w. When only the initializer is forbidden, use

$$F(w) := \big(I(Sn, w; v.h) = h[I(n, w; v.h)/v]\big).$$

When only the predecessor is forbidden, use

$$F(w) := \big(I(Sw, a; v.h) = h[I(w, a; v.h)/v]\big).$$

When both are forbidden, use

$$F(w) := \big(I(Sw, w; v.h) = h[I(w, w; v.h)/v]\big).$$

In each case $F(C)$ is a permitted successor computation. Transport along $C = 0$ gives the exact desired equation $F(0)$. Choose w fresh for h and all passive parameters, and rename the bound state variable before forming each motive. Hence only the displayed argument positions are changed; unrelated zero occurrences and passive parameters stay fixed. This is a construction of individual proofs by the primitive transport rule, not an assumed operation substituting terms into whole proofs. □

2.3 Exact preservation of native sequents

Theorem 2.4 (Witness-selected introduction)**.** *For every finite native context* Γ *and object formula* A*,*

$$\Gamma \vdash_{\mathcal{C}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}_{\mathrm{w}}} A.$$

Both translations are effective transformations of finite derivations. The terms, formulas, contexts, and final sequent are unchanged. In particular the new rule has the original closure interpretation, not a new progressive interpretation of atomic paired formulas.

Proof. Every application of DI_{w} is literally an old progress application or an old case application. The common rules are unchanged. Thus a new derivation is already an old derivation, proving one inclusion without any translation of formulas.

For the reverse inclusion, induct on the old derivation, recursively transforming its premise derivations first. Common rules are repeated unchanged. Four introduction situations exhaust the remaining cases.

1. Old case, with compound matrix or nonliteral-zero witness. If B is compound, or if $t \not\equiv_{\mathrm{syn}} 0$, then $\delta_{B,t}(z) = 0$. The transformed premise is exactly a premise of DI_{w}, so repeat the introduction.

2. Old case, with literal equation matrix and literal-zero witness. Write the transformed premise as

$$\pi : \qquad \Gamma, E(0,0) \vdash E(0, Sz).$$

Choose its eigenparameter z fresh for all retained data. Under the new context $\Gamma, E(C,0)$, transport the hypothesis $E(C,0)$ along the closed theorem $C = 0$, using the fixed motive $E(w,0)$. This gives $E(0,0)$. Lemma 2.1 inserts this derivation at the uses of the assumption $E(0,0)$ in π, giving

$$\Gamma, E(C,0) \vdash E(0, Sz).$$

Transport the resulting conclusion along $0 = C$, now with motive $E(w, Sz)$. The conclusion is

$$\Gamma, E(C,0) \vdash E(C, Sz).$$

The term C is closed and syntactically nonzero. Consequently DI_{w} with witness C selects the case premise just obtained and concludes the original formula $D_{x,y}E(x,y)$. The conclusion's syntax does not record which witness was used.

This argument changes the retained ordinary assumption by proved transport and cut. It does not apply a numerical substitution to π, so it cannot accidentally change the selected mode of an introduction inside that derivation.

3. Old progress, with literal-zero witness. Its matrix is a literal equation and its witness is the literal zero term. Therefore DI_{w} selects progress and repeats the old inference using the recursively transformed premise.

4. Old progress, with a nonliteral-zero witness. Write its transformed premise as

$$\pi : \qquad \Gamma, E(t,z) \vdash E(t,Sz), \qquad t \not\equiv_{\mathrm{syn}} 0.$$

Choose a new variable u absent from Γ, t and the entire matrix. Consider the equation template $\widetilde{E}(u,y) := E(t,y)$, whose u coordinate is unused. DI_{w} with witness 0 selects progress and uses π directly to give

$$\Gamma \vdash D_{u,y}E(t,y).$$

Paired elimination of this formula introduces the local closure $Q_yE(t,y)$. Its fresh existential parameter is absent from this closure, because u was unused.

Under $\Gamma, Q_yE(t,y), E(t,0)$, hypothesis closure and QE give $E(t,Sz')$, for a newly chosen parameter z' fresh for all retained context and passive data. Since t is syntactically nonzero, DI_{w} with witness t selects cases and proves

$$\Gamma, Q_yE(t,y) \vdash D_{x,y}E(x,y).$$

The paired elimination now discharges its local closure and gives exactly the original conclusion. The original premise proof π has been inserted once. Neither its witness nor its numerical parameters have been substituted.

All fresh parameters used in these four constructions can be chosen outside the finitely many variables of the transformed premises. Renaming preserves the deterministic guard by Lemma 2.1. This completes the induction on the old derivation and proves the second inclusion. □

Substitution and arithmetic repairs are consequences, not premises. Numerical substitution can change a witness from a variable into the literal constant 0, and can also create a forbidden literal-zero initializer in a restricted computation. It is therefore not valid to claim that arbitrary numerical substitution preserves every primitive new rule instance. Once Theorem 2.4 is proved, the inherited admissible substitution transformation may be performed in the old calculus and its result translated back. Likewise, any separately established repair of the restricted computation axioms transfers through the identity on native sequents. The proof above used neither assertion circularly.

Consequences and limits. The theorem transfers the inherited whole-domain bi-interpretation with full first-order intuitionistic PRA, and its equiconsistency, to $\mathcal{C}_{\mathrm{w}}$: compose the inherited proof translations with the two identity-sequent transformations just proved. All formula round trips and numerical interpretation composites remain those of the old calculus. The new contribution is a restriction of primitive proof choices, rather than a different numerical algebra or a new meaning of intuitionistic logic. No absolute minimum, lower-bound optimality, or universal bound on the shortest certificates follows. A concrete checker-size claim additionally requires the complete implementation, its raw-input soundness argument, and measured source.

Attribution and the scope of the novelty search. The unused-witness progress construction is already present in the inherited Choice-DI proofs, and the closed term C and its direct zero certificate are already present in the certified-zero draft. Existing progressive-binder proposals also use deterministic introduction, but change the atomic closure interpretation and the corresponding elimination rules.

The construction here combines the first two inherited mechanisms to select the old case/progress rules by witness syntax while preserving every old native sequent. Searches of the seventeen current extracted archives and their included text/code references did not locate this exact restriction and simulation. That bounded search is not a claim of literature-wide priority.

A general form of the normalization. The argument does not depend on the numerical value zero having a special induction-theoretic role in the witness coordinate. If p is any fixed closed term and there is a syntactically different closed term q with proofs of $p = q$ and $q = p$ in the common fragment, progress may instead be selected exactly at witness p. Repair a forbidden case witness p using q, and introduce every auxiliary unused-witness progress formula with witness p. The rest of the simulation is identical. Zero is chosen here because its syntax is the cheapest recognizable witness and its nonliteral representative C has a direct computation certificate. The induction coordinate itself still begins at zero; only the independent existential witness coordinate is being normalized.

3 The complete checker and its exact trust boundary

This is the complete 555-byte source. The explanatory expansion in `code/readable_checker.py` is independently executable. Neither program imports a producer, a theorem database, a numerical evaluator, or the other checker.

```
import json
k = 0
i = lambda a, t=0j, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d else
     d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or t == len(a) % 2 == 0) and k)])
q = json.load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

3.1 Certificate grammar and sorted interpretation

Input is a JSON array. Its last element is the requested closed formula; earlier four-field packets $[m, s, u, t]$ are consumed backwards. An unused prefix is inert. Success means exit zero and exactly `ACCEPT` followed by a newline. An exception rejects with a nonzero exit status. The program does not use Python `assert`, so optimized Python does not remove a logical check.

The arity representation is inherited [3, 2]:

Array arity	Numerical interpretation	Formula interpretation
0	Zero	Truth $0 = 0$
1	Successor, with two dummy slots	Paired binder D
2	Zero; annotations ignored	Equality
3	Scalar iterator, one state slot	Closure Q of its last field
At least 4	Zero; children ignored	Truth; children ignored

Canonical equations are $[s, t]$, paired formulas are $[B]$, and closures are $[B, B, B]$. Numerical zero is scalar 0, successor is unary, and an iterator is ternary. Numerical roots have depth three; formula roots have depth zero. Each traversal adds $5 - \text{arity}$ to the depth. At a numerical root, scalar positions $0, 1, 2$ are dummy zeros; successive protected slots use pairs $3, 4$, then $5, 6$, and so on. Negative allocated labels denote rigid parameters. In a successor its child has two dummy zero slots; an iterator's counter and seed have one dummy slot, and the update has its actual state slot. In D, the two slots are y, x; in Q, the one slot is y.

The opener retains a scalar in $k \leq a < d$. An escaping numerical coordinate is filled by an image checked at numerical root depth three; escaping formula padding erases to truth. The default image $0j$ is intentionally not ordered against a real numerical coordinate. If it is actually needed as a missing numerical image, the comparison fails, just as it fails for the old `None`. Unused images need not be evaluated. The list predicate `a * 0 == []` recognizes lists among JSON values on which the expression succeeds; unsupported types reject. The explicit-list variant is provided for a clearer spelling.

At ignored high arities the decoder is constant. The appropriate invariant is semantic opening at meaningful sorted positions, conditional on a successful traversal. A global raw-tree idempotence assertion is not assumed. The inherited opening argument and its complete source are identified in the provenance record; the new checker changes none of its depth arithmetic or numerical substitution.

3.2 The three operations and the shared continuation

For a truthy major m, the checker proves m in the retained context. If $m = [s, u]$ and the target opening of t is the current goal, it reduces that goal by equality transport. If s is falsy and $m = [t, t, t]$, it performs closure specialization from its zero instance. A failed endpoint match keeps the original goal. A unary paired major exports its tripled matrix as a closure; other nonempty arrays export constant truth. The recursive major proof is always required.

For a falsy major and a truthy initializer u, the packet generates the equation

$$H = \begin{cases} I(0, u; v.t) = u, & s \text{ falsy}, \\ I(Ss, u; v.t) = t[I(s, u; v.t)/v], & s \text{ truthy}. \end{cases}$$

Every selected generated equation is a nonempty two-element array. Canonical forbidden zero seeds or zero predecessors are restored by the certified-zero computation lemma. No separate arithmetic opcode or semantic test that an initializer is positive is required.

For a falsy major and falsy initializer, the packet introduces a paired goal, with t as its witness. For canonical nonempty goals, unary unpacking checks the paired shape. Once the witness is opened, progress is selected exactly by `t == len(a) % 2 == 0`. On canonical data this says that the witness is literal zero and the opened matrix is an equation. Compound matrices use their base instance. Larger harmless truth aliases retain the inherited constant interpretation.

The continuation allocates a fresh negative name, opens the successor goal, appends exactly one local hypothesis, and recurses with a new context list. The context is not shared mutably with a major subproof. Already checked goals and contexts do not mention names admitted only after this allocation.

JSON fractional numbers in a newly admitted interval must be treated as distinct fresh names, not identified with its integer endpoint; only finitely many occur in a finite trace.

3.3 Why Boolean dispatch is legitimate

The old assignment was a conditional on m, followed by a conditional on u. In a successful new execution, a truthy permitted m has a truthy triple $m * 3$. If no major is selected, a selected computation has a nonempty equation array. Therefore the expression

$$m * 3 \texttt{ or } (u \texttt{ and } H) \texttt{ or introduction}$$

selects precisely the intended branch. Some old falsy marker aliases, including null, fail at multiplication. Canonical arithmetic null markers are replaced by []. Both old markers are falsy and compare unequal to an integer parity, so this replacement preserves the old Choice-DI program's continuation. This is a restriction and normalization of certificate data, not an additional logical inference.

3.4 An exact simulation of every new accepting trace

For a false-major packet executed by the witness checker, change its marker to 0 if its raw field t equals 0, and to [] otherwise. Leave every true major unchanged. Apply this fixed transformation to the complete finite input stack. For a false major, the old guard

```
len(a) % 2 == m
```

then has exactly the new guard's value

```
t == len(a) % 2 == 0.
```

Indeed the first transformed marker compares equal only at parity zero, and the second never compares equal to integer parity. This remains true for arithmetic packets: their update occupies t, and the same opening image is obtained. Both major branches, every recursive call, and every new local assumption otherwise coincide. Replacing the unused invalid-image default $0j$ by `None` cannot change a successful trace, because either default raises if used as a numerical image. Thus a new finite acceptance simulates an old certified-zero acceptance.

This argument includes noncanonical successful inputs. It avoids the false assertion that every generated $H(b)$ is a uniformly true arithmetic equation. In general only $H(\vec{0})$ is known. Freshness permits substitution of zero for all newly introduced names in a continuation proving a conclusion independent of them, followed by cut against the known zero instance. This substitution and cut take place in the simulated old calculus. This is the inherited supply argument, not a new reflection rule.

For completeness, first use the theorem in Section 2 to normalize a finite named derivation. Repair excluded computation leaves by the certified zero. Serialize canonical terms and formulas with fresh names and use [] for falsy packet markers. In a canonical computation all meaningful parameters are already allocated, so opening at zero or at the new name does not alter the supplied equation. The canonical introductions now have exactly the mode selected by their witness. This constructs an accepting finite certificate, assuming sufficient computation resources.

4 Why this is full intuitionistic PRA

The ordinary realization has actual equality, genuine falsity, the whole numerical domain, the iterator equations, contextual literal-equation induction, numerical coverage, and separation $(0 = S0) \to \bot$.

The native formula interpretation is

$$(s = t)^\circ = (s = t), \quad (D_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)), \quad (Q_y B)^\circ = B^\circ(0) \to \forall y B^\circ(y).$$

The deterministic witness restriction does not change these meanings. Its introductions are old case/progress instances, and its converse simulation preserves the same end formulas and retained hypotheses.

For orientation, the inherited scalar selector is $C(n, a, b) = I(n, a; v.b)$. Its zero and successor computations make it a formula gate and a selector between arbitrary formulas after padding their binder depths. The paired binder then defines implication, conjunction, disjunction, existential quantification, and universal quantification; $0 = 1$ represents falsity. Those definitions derive the ordinary intuitionistic rules. Their explicit formulas, all introduction and elimination arguments, and both logical composites are reproduced in Appendix A. In particular the native paired formula is interderivable with

$$\exists_* x(B(x, 0) \Rightarrow_* \forall_* y B(x, y)).$$

This round trip is essential; a progressive reinterpretation of atomic D would not suffice.

The numerical bridge follows the proved dependency order:

1. Equation induction and cases construct a scalar predecessor P and prove $P0 = 0$, $P(Sn) = n$ internally.
2. These yield subtraction and a numerical test reflecting actual equality on the whole domain, rather than merely on standard values.
3. A characteristic-term construction transfers induction from literal equations to all fixed quantifier-free formulas.
4. Pairing and projections, established using that induction, compile each fixed general primitive recursion into scalar iteration.

The full inherited proofs are reproduced in Appendices B, C, and D; they are not counted as imported checker code.

Conversely, iPRA defines every fixed scalar iterator by primitive recursion that ignores its counter argument in the update. For each finite description, the double translation satisfies the original defining equations. Equation induction proves uniqueness of its values, at arbitrary internal counters and with passive parameters fixed. Thus both numerical composites are internally isomorphic to the original domain by $x = y$. There is no restriction to a cut, retract, quotient, or standard numerals. Both interpretations preserve falsity, using the explicit ordinary separation sentence. Effective proof translations give equiconsistency; no self-consistency theorem is claimed. Appendix E spells out these composites.

5 Reproducibility, evidence, and remaining scope

The new named proof normalizer is separate from the trusted checker. It records the two actual mode repairs, preserves the final formula, and then uses the unchanged attributed arity emitter. Its binder convention uses globally distinct names; mathematical alpha-renaming provides that convention for an arbitrary finite derivation.

The focused witness suite contains 18 positive and five negative cases. It tests the three compact exact-list implementations and the readable expansion, in ordinary and optimized Python: 184 complete

CLI runs. Two further mutation controls remove the literal-equation restriction; the mutated program then accepts a forged compound-progress certificate, whereas the delivered programs reject it. A zero-case test uses its base assumption essentially, and a nonzero-progress test uses its induction hypothesis with the witness occurring in the matrix. Their untranslated certificates reject and the normalized certificates accept. These examples exercise the two proof transformations, rather than only vacuous witnesses.

The unchanged-choice checker replays the 36 supplied certified-zero cases after its marker translation, in both Python modes: 72 executions. The independent initial baseline audit also recorded 216 executions of the three upstream programs. Appendix F has six separate symbolic congruence certificates and three side-condition rejection controls. These suites overlap in mathematical content and are not counts of distinct theorems. Finite checks support the implementation; the universal soundness and equivalence conclusions rest on the written arguments.

From a fresh extraction, run:

```
python3 reproduce.py
python3 code/checker.py < examples/zero_case_uses_base.json
python3 code/checker.py < examples/nonzero_progress_uses_IH.json
python3 code/checker.py < examples/closed_false.json
python3 build.py
```

The first two individual proof checks accept; the last rejects. Reproduction verifies package hashes before rerunning tests. Rebuilding or intentionally editing the source can change those hashes, particularly PDF metadata, so use a working copy if the original manifest is to remain an identity record. The build produces a standalone LaTeX source and compiles without shell escape. The only build dependency outside Python's standard library is a suitable TeX installation.

The report and code establish a small positive result under a fixed, reproducible comparison. They do not establish the significant breakthrough requested. Removing an independent certificate choice can simplify a rule inventory while increasing another implementation cost: here its guard adds five bytes. The smaller overall programs should therefore not be advertised as evidence of a radically simpler numerical foundation.

Attributed complete bridge proofs

The following five sections reproduce the corresponding source fragments from the supplied certified-zero package, byte-for-byte. They document the inherited theory to which the new identity-sequent theorem applies. Their historical constructions are not new contributions of this report.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \;\; \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \;\; \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \;\; \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [5] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

B.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0+b=b$ and $Sa+b=S(a+b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a+a$ and $3a$ means $a+a+a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n,0;u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n,0;u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

B.2 A ramp with a global scaling identity

Define

$$G(n) = I(n,0;u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0)=1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n)+2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

B.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

B.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

B.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

C From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [4]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix B, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

C.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In

particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

C.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch

of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

C.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$

$$(Sx) + y = S(x + y), \tag{32}$$

$$d(Sx, Sy) = d(x, y), \tag{33}$$

$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

C.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b+1 = S(b+0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}(K(d(Sa, b))). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, (a = b \vee (a = b \to \bot)) \tag{41}$$

is a theorem, not an extra hypothesis.

C.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

C.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

D Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [6]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

D.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

D.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

D.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

E Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

E.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem E.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse

direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

F A narrower congruence rule and its exact boundary

This section strengthens the supplied report *Counter Congruence Suffices for iPRA*, specifically its `theory/counter_theorem.tex`, in `ipra_counter_congruence_2026-11-10.zip` at ProveIt commit `76a2651601d17b34d46753f189ea026a21ce8d29`. The numerical basis, paired logical calculus, root-replacement equality algebra, and formula lifting are inherited. The refinement is that counter congruence needs only a literal-zero seed and updates independent of the changing state. A shorter representation reduces its numerical-congruence proof overhead. A second, distinct rule permits an even shorter derivation. Finally a rule countermodel shows why restricting the premise endpoints to successors is not the same simplification.

These are mathematical inference results. No smaller checker implementing these bases is claimed. The supplied source and documentation use the MIT No Attribution License, copyright 2026 ProveIt Contributors; that license is retained with the unchanged reference checker and producer. Mathematical attribution is retained regardless of the license's permission.

F.1 Complete calculus used by the refinement

There is one numerical sort. Finite terms are generated by variables, 0, S, and $R(n, a; u, v.h)$. Only the body h binds u, v: u is the old counter and v is the previous state. Other variables are passive parameters. Successor may instead be an abbreviation $Sa := R(0, a; u, v.0)$, with fresh unused u, v. For a conventional first-order signature, introduce one function symbol per fixed, finite earlier-stage body description; no variable ranges over functions or descriptions, and there is no uniformly total evaluator. The computation schemas are

$$R(0, a; u, v.h) = Sa, \qquad R(Sn, a; u, v.h) = h[n/u, R(n, a; u, v.h)/v]. \tag{R0, RS}$$

The substitutions are simultaneous and capture-free. In particular, the scalar-state iterator from the smallest measured checker is not being silently used here: this R body receives *both* counter and state.

Object formulas and internal resources have the grammars

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \mathsf{Q}_y A(y).$$

The variables x, y of D are distinct. Contexts Γ are finite reusable lists of object assumptions and internal resources. An object hypothesis has the one-node rule $\Gamma \vdash A$ when $A \in \Gamma$. An internal lookup, written $\Gamma \triangleright \mathsf{Q}_y B$, also has one node when that resource belongs to Γ; it is not an object-formula conclusion. R0 and RS are one-node object axioms in every retained context. The logical rules are exactly

$$\frac{\Gamma \triangleright \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \quad (\mathsf{QE}),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(a, y) \vdash F}{\Gamma \vdash F} \quad (\mathsf{DE}),$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad (\mathsf{DI}_0), \qquad \frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)} \quad (\mathsf{DI}_1).$$

The progress matrix E must be one literal equation, whereas B may be any object formula. In DE, a is fresh for the retained context, conclusion, and fixed parameters of B. In either introduction, z is fresh for the context, witness t, and fixed matrix parameters. Local assumptions are discharged. Weakening, exchange, contraction, renaming, and capture-free numerical substitution are admissible structural operations, not additional proof nodes.

The equality rule retained in every version below is root-left replacement:

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r} \quad (\mathsf{RL}).$$

Both displayed premises may be arbitrary proofs. Restricting its major to a retained hypothesis is *not* part of the result. The proposed congruence rule is

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash R(s, 0; u, v.h(u, \vec{p})) = R(t, 0; u, v.h(u, \vec{p}))} \quad (\mathsf{ZCC}),$$

where v has no free occurrence in the displayed body, including below nested numerical binders. Its seed is literally 0, not merely a term proved equal to zero. There are no primitive reflexivity, general congruence, transport, coverage, separation, or additional induction rules. Call the resulting calculus $\mathcal{C}_0$.

F.2 Equality and cases before using ZCC

Lemma F.1 (Root equality algebra)**.** *RL and computation derive reflexivity, symmetry and transitivity. From proofs of $L = R$, $L = F$, and $R = G$, exactly two additional RL nodes produce a proof of $F = G$.*

Proof. Set $c_s = R(Ss, 0; u, v.u)$. RS gives $c_s = s$. Using two copies as the major and minor of RL gives $s = s$, in three nodes. An available proof of $s = t$ and this reflexivity give $t = s$. For transitivity, reverse $s = t$ to $t = s$ and apply RL with minor $t = r$. For the last statement, RL applied to $L = R$ and $L = F$ gives $R = F$. Apply RL again with that major and minor $R = G$ to obtain $F = G$. □

Lemma F.2 (Numerical cases)**.** *For every object template $A(y)$, from $\Gamma \vdash A(0)$ and $\Gamma \vdash A(Sz)$ one can derive $\Gamma \vdash A(t)$, where z is fresh for the retained data and t. The construction uses neither equality rule nor progress introduction and adds four nodes to the sum of the two premise trees.*

Proof. Choose x unused in A. Weaken the successor proof by $A(0)$ and apply case introduction with witness 0, producing $\mathsf{D}_{x,y}A(y)$. Eliminate it, obtaining the resource $\mathsf{Q}_y A(y)$ because the witness coordinate is unused. Look up that resource and use QE with the original base proof, weakened to this context. Close DE. The added nodes are DI, DE, resource lookup, and QE. □

F.3 Zero seed and no state dependency suffice

Define the fixed numerical context

$$U(n) := R(n, 0; u, v.S(Su)).$$

Its two direct computations are

$$U(0) = S0, \qquad U(Sz) = S(Sz).$$

Numerical cases therefore prove

$$U(n) = Sn \tag{U}$$

for arbitrary n, with arbitrary retained context. This proof has six nodes and uses no congruence or induction rule.

Theorem F.3 (Numerical congruence from ZCC)**.** *For every fixed numerical template $F(x, \vec{p})$, $\mathcal{C}_0$ admits*

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash F(s, \vec{p}) = F(t, \vec{p})}.$$

Its displayed construction adds exactly twenty nodes to the supplied equality proof and introduces no progress inference.

Proof. Freshen all new binders away from the passive parameters and source terms. Apply ZCC to $s = t$ through U. It is permitted because U has seed zero and its update $S(Su)$ ignores v. Thus $U(s) = U(t)$. The two representation proofs (U) give $U(s) = Ss$ and $U(t) = St$. Two root replacements, as in Lemma F.1, give

$$Ss = St.$$

For the arbitrary template F, set

$$V_F(n) := R(Sn, 0; u, v.F(u, \vec{p})).$$

Its update ignores its state. A single RS instance proves

$$V_F(n) = F(n, \vec{p}) \tag{V}$$

for every n. No cases or equality substitution are hidden in this computation. Apply ZCC to the derived $Ss = St$ with body $F(u, \vec{p})$; its conclusion is $V_F(s) = V_F(t)$. The two endpoint computations (V), followed by two more root replacements, give $F(s, \vec{p}) = F(t, \vec{p})$.

For a source proof with P nodes, the successor stage has $(P + 1) + 6 + 6 + 2 = P + 15$ nodes: one ZCC, two six-node representations, and two RL nodes. The final stage adds one ZCC, two one-node computations, and two RL nodes. The total is $P + 20$. Each supplied premise occurs once. □

The supplied report's displayed numerical-congruence construction has $P + 56$ nodes. Merely replacing its final symmetry/transitivity assembly by the two-root-replacement lemma would improve that existing construction to $P + 44$. The present zero-seed representation gives $P + 20$. Thus the comparison separates the new representation from a shorter assembly of old proofs. These are exact sizes of specified proof trees, not minimum proof-size claims. Their term labels contain F and may grow with its size.

F.4 Formula transport and full consequence equivalence

Theorem F.4 (Full transport)**.** *For every native Eq/D template $A(w)$, $\mathcal{C}_0$ admits*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash A(s)}{\Gamma \vdash A(t)}.$$

Its expansion uses no additional progress induction.

Proof. Use external induction on the finite number of paired binders in A, uniformly in all retained contexts. For $A(w) \equiv P(w) = Q(w)$, obtain $P(s) = P(t)$ and $Q(s) = Q(t)$ by Theorem F.3, then use root equality algebra to join them with $P(s) = Q(s)$.

For $A(w) = \mathsf{D}_{x,y}B(w, x, y)$, eliminate $A(s)$ at a fresh a. The retained resource is $\mathsf{Q}_y B(s, a, y)$. Introduce the desired $A(t)$ by case introduction with witness a and fresh z. Its local assumption is $B(t, a, 0)$. Symmetry gives $t = s$, so the external induction hypothesis transports that assumption

backwards to $B(s, a, 0)$. QE then supplies $B(s, a, Sz)$. A forward transport by the same external induction hypothesis gives $B(t, a, Sz)$. Close case introduction and DE. Both recursive calls concern the strictly smaller matrix B. Freshness conditions are met by renaming. This is structural reasoning about finite formulas, not compound arithmetic induction. □

Corollary F.5 (Identity-language equivalence)**.** *Replacing the supplied unrestricted counter-congruence rule by ZCC leaves every native sequent derivable exactly when it was derivable before. The same holds when comparing with the supplied unrestricted object-transport calculus. Therefore the inherited whole-domain bi-interpretation and equiconsistency with full first-order iPRA transfer.*

Proof. Every ZCC inference is an old counter-congruence inference. Conversely, Theorem F.3, with context $R(x, a; u, v.h)$, recovers every old counter-congruence instance, including those with a state-dependent body and arbitrary seed. Theorem F.4 recovers full object transport. In the other direction, RL is full transport at the left root, while ZCC follows by transporting one occurrence in a reflexivity equation. The transformations recursively preserve end sequents and freshness. The arithmetic and logical iPRA bridge is the attributed supplied bridge; this refinement introduces no new domain or equality. □

F.5 A different rule with only five nodes of overhead

Define a separate primitive rule

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash R(Ss, 0; u, v.h(u, \vec{p})) = R(St, 0; u, v.h(u, \vec{p}))} \quad (\mathsf{ZCC}^+),$$

again with no free occurrence of v in h. Its premise endpoints are arbitrary terms; the rule itself wraps them in successors. This is a changed rule, *not* ZCC with a side condition on its premise.

Proposition F.6. *Replacing ZCC by* ZCC^+ *also gives the full native consequence relation. Arbitrary numerical congruence now has a* $P + 5$*-node construction from a* P*-node source equality, without cases or progress induction.*

Proof. For arbitrary F, apply ZCC^+ using body $F(u, \vec{p})$. This directly gives $V_F(s) = V_F(t)$. Two RS endpoint computations and two RL nodes give $F(s) = F(t)$, adding $1 + 1 + 1 + 2 = 5$ nodes. Formula lifting is unchanged. Conversely, Theorem F.3 in $\mathcal{C}_0$ derives ZCC^+ by choosing context $R(Sx, 0; u, v.h)$. Thus the two full calculi are equivalent. □

This factors numerical congruence through a fixed successor-computation shape. It is not claimed to remove the cost of numerical binding or substitution: that cost remains in the computation rules. No source-size figure is assigned to this alternative calculus.

F.6 Why successor-only premise endpoints are insufficient

Consider instead retaining ZCC only when its *premise* is syntactically $Sp = Sq$, so its conclusion is $R(Sp, 0; h) = R(Sq, 0; h)$. Keep all other rules unchanged and supply no rule wrapping arbitrary premise endpoints.

Proposition F.7 (A rule countermodel)**.** *This successor-premise-only restriction does not derive full numerical congruence and does not preserve the old native consequence relation. In particular it does not derive*

$$0 = S0 \ \vdash \ S0 = S(S0).$$

Proof. Use the standard numerical domain $\mathbb{N}$. Interpret $0, S, R$ by the ordinary zero, successor, and successor-seeded primitive recursion. Interpret a native equation $s = t$ by

$$e(s^{\mathbb{N}}, t^{\mathbb{N}}), \qquad e(m, n) \iff m = n \text{ or } m, n \in \{0, 1\}.$$

For avoidance of ambiguity, this is a *rule interpretation* of the native equation atom; it is not asserted to be a model of ordinary first-order logic with that atom as actual equality. Interpret resources and paired formulas by their ordinary classical meanings

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ(y), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

Classical truth suffices for a rule-soundness countermodel of derivability.

The relation e is an equivalence relation, so RL is sound. Every computation equation has actually equal values and therefore is e-true. For the restricted congruence rule, both counter values are positive. On positive numbers e agrees with actual equality. Consequently its true premise implies identical actual counters, and the recursor values agree. In fact this argument would allow arbitrary seeds and updates whenever both premise counters are successors.

Hypothesis use, resource lookup, QE and DE are sound by their displayed meanings. Case introduction is sound by standard numerical coverage. Progress introduction is sound because ordinary induction on standard $\mathbb{N}$ holds for the interpreted predicate, including predicates whose equation atoms use e; only its actual truth set is relevant. Freshness and substitution have their usual numerical meanings. Thus every rule of the restricted calculus preserves truth in this interpretation.

The premise $0 = S0$ is true since $e(0, 1)$ holds, while the conclusion is false since $e(1, 2)$ does not. Hence that sequent is underivable. The old calculus derives it by successor congruence, proving the claimed strict loss of native consequences. □

This countermodel rules out the proposed identity-language restriction. It is not a theorem excluding every possible alternate interpretation, changed equality convention, or enlarged axiom system.

F.7 Evidence, overlap, and limits

The source screen recursively inspected 32 distinct ZIP payloads from the 17 current archives, with 867 distinct text files and 443 parsed Python ASTs. It found the expected old state-free-context remark but no explicit zero-seed congruence restriction, wrapped rule, or successor-premise countermodel. Pattern screening does not decide semantic duplication or establish literature-wide priority. The result is attributed as a refinement of the existing counter-congruence argument, not an invention of its arithmetic or logical basis.

The supporting script `validate_zero_seed.py` preserves the unchanged upstream readable checker and intercepts every CC call to require literal-zero seed and absence of the outer state slot, correctly tracking nested numerical binders. It constructs three symbolic proofs (successor, initializer, and nested-update congruence), each with a one-node source and exactly 21 nodes. A separate, explicitly coded wrapped-rule branch verifies the corresponding three six-node proofs. All six pass. Controls with a nonzero seed, a direct state occurrence, and a state occurrence below another recursor are rejected. A finite check of the countermodel relation is supplied only as a sanity check; the universal rule-soundness argument is the preceding proof.

These artifacts verify concrete proof trees and side conditions. They are not Lean/Rocq formalizations, universal correctness proofs of a new checker, or measurements below the 558-byte current frontier. The numerical proof bounds do not bound the size of formula-transport lifting, serialized term labels, or runtime. No absolute simplest axiomatization or significant checker-size breakthrough follows from this appendix alone.

References

[1] Vladimir Reshetnikov, *ProveIt, current draft corpus*, commit `76a2651601d17b34d46753f189ea026a21ce8d29`, 10 November 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/76a2651601d17b34d46753f189ea026a21ce8d29/docs/drafts.

[2] *A Certified Zero Removes a Computation Opcode*, `ipra_certified_zero_2026-11-10.zip`, in [1]. Source of the selected comparison programs, certified-zero computation repair, named emitter, and reproduced complete arithmetic/logical bridge.

[3] *An Arity Calculus for iPRA*, `ipra_arity_calculus_2026-11-10.zip`, in [1]. Source of the arity encoding and sorted-opening argument, as inherited by [2].

[4] Supplied actual-equality reflection proof, `reference/equality_reflection.tex` in [2], with its attribution to the earlier forward-left development.

[5] Supplied scalar predecessor proof, `reference/scalar_predecessor.tex` in [2], with its attribution to the earlier scalar and unparenthesized one-hole developments.

[6] Supplied pairing and primitive-recursion proof, `reference/primitive_recursion.tex` in [2], with its attribution to the earlier progress-closure development.

[7] *Counter Congruence Suffices for iPRA*, `ipra_counter_congruence_2026-11-10.zip`, in [1]. Source of the different R calculus refined in Appendix F.